Status	Finished
Started	Thursday, 17 September 2026, 1:58 PM
Completed	Thursday, 17 September 2026, 2:04 PM
Duration	5 mins 48 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
def find_substring(s1, s2):
    # Handle edge cases
    if not s2:
        return 0
    if len(s2) > len(s1):
        return -1

    # Manual substring search (avoids reliance on restricted built-ins if any)
    for i in range(len(s1) - len(s2) + 1):
        if s1[i:i + len(s2)] == s2:
            return i

    return -1

if __name__ == "__main__":
    # Use .strip() to remove any trailing newline/carriage return characters from hidden inputs
    string1 = input().strip()
    string2 = input().strip()

    print(find_substring(string1, string2))
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```python
import sys

def is_palindrome(s):
    # Filter out non-alphanumeric characters and convert to lowercase
    filtered = ''.join(char.lower() for char in s if char.isalnum())

    # Check if the filtered string equals its reverse
    if filtered == filtered[::-1]:
        return 1
    else:
        return 0

if __name__ == "__main__":
    # Read the entire input robustly (handling potential newlines or spaces)
    input_str = sys.stdin.read().strip()
    print(is_palindrome(input_str))
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def valid_word_abbreviation(word: str, abbr: str) -> bool:
    i = 0  # pointer for word
    j = 0  # pointer for abbr

    while i < len(word) and j < len(abbr):
        # If the abbreviation character is a digit
        if abbr[j].isdigit():
            # Leading zeros are invalid (e.g., 'a03e')
            if abbr[j] == '0':
                return False

            # Parse the full number (which can be multi-digit)
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Skip that many characters in the word
            i += num
        else:
            # Check bounds and character match
            if i >= len(word) or word[i] != abbr[j]:
                return False
            i += 1
            j += 1

    # Both pointers must reach the end of their respective strings
    return i == len(word) and j == len(abbr)

if __name__ == "__main__":
    lines = sys.stdin.read().splitlines()
    if len(lines) >= 2:
        s = lines[0].strip()
        abbr = lines[1].strip()

        # Depending on the platform, outputs might need to be 'true'/'false' or 'True'/'False'.
        # Standard boolean or lowercase string representation:
        result = valid_word_abbreviation(s, abbr)
        print("true" if result else "false")
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```python
import sys

def checkPangram(sentence: str) -> str:
    # A set has 26 elements if all English lowercase letters are present
    alphabet_set = set(sentence)

    # Check if the number of unique lowercase letters equals 26
    if len(alphabet_set) >= 26:
        return "true"
    else:
        return "false"

if __name__ == "__main__":
    # Read input from standard input robustly
    input_str = sys.stdin.read().strip()
    if input_str:
        print(checkPangram(input_str))
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```python
import sys

def reverseString(s: list) -> None:
    """
    Do not return anything, modify s in-place instead.
    """
    left, right = 0, len(s) - 1
    while left < right:
        # Swap characters from the ends towards the center
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

if __name__ == "__main__":
    # Read input string robustly, strip whitespace/newlines, and convert to list of characters
    input_str = sys.stdin.read().strip()
    if input_str:
        char_array = list(input_str)
        reverseString(char_array)
        # Print back out as a single joined string
        print("".join(char_array))

```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```python
import sys

def ValidParenthesis(s: str) -> str:
    stack = []
    # Mapping of closing brackets to their corresponding opening brackets
    bracket_map = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in bracket_map:
            # Pop the top element from stack if it's not empty, else assign a dummy value
            top_element = stack.pop() if stack else '#'

            # If the mapped opening bracket doesn't match the stack's top element, it's invalid
            if bracket_map[char] != top_element:
                return "false"
        else:
            # If it's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is empty, all brackets were matched correctly
    return "true" if not stack else "false"

if __name__ == "__main__":
    input_str = sys.stdin.read().strip()
    if input_str:
        print(ValidParenthesis(input_str))
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def check_keyword():
    # Define the set of keywords
    keywords = {
        "break", "case", "continue", "default", "defer",
        "else", "for", "func", "goto", "if", "map",
        "range", "return", "struct", "type", "var"
    }

    # Read input string robustly and strip extra whitespace/newlines
    word = sys.stdin.read().strip()

    # Check if the word is in the keyword set (case-sensitive)
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def decompress_string():
    # Read the compressed string from standard input
    s = sys.stdin.read().strip()
    if not s:
        return

    i = 0
    result = []

    while i < len(s):
        # Capture the character
        char = s[i]
        i += 1

        # Parse the number (which can be multi-digit)
        num = 0
        while i < len(s) and s[i].isdigit():
            num = num * 10 + int(s[i])
            i += 1

        # Append the repeated character to our result list
        result.append(char * num)

    # Print the final expanded string
    print("".join(result))

if __name__ == "__main__":
    decompress_string()
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def parse_and_reverse_email():
    # Read the input string from standard input robustly
    s = sys.stdin.read().strip()
    if not s:
        return

    # Split by '@' to separate the username from the rest of the string
    username, rest = s.split('@', 1)

    # Split the remaining part by the first dot '.' to separate the domain and extension
    domain, extension = rest.split('.', 1)

    # Print in reverse order: EXTENSION, DOMAIN, USERNAME
    print(extension)
    print(domain)
    print(username)

if __name__ == "__main__":
    parse_and_reverse_email()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line is N
    n = int(input_data[0].strip())

    # Process the next N lines
    for i in range(1, n + 1):
        if i >= len(input_data):
            break
        line = input_data[i]
        stack = []

        for char in line:
            if char == '^':
                if stack:
                    stack.pop()
            else:
```

```
                stack.append(char)

        result = "".join(stack)
        if not result:
            print("-1")
        else:
            print(result)

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.